Scene-Graph Robot - Codebase Walkthrough

Section 3.1: Task-Driven Scene Graph (HODOR, CVPR 2026)

September 14, 2026

1. Purpose and scope

This report is an inside-out walkthrough of the `scene-graph-robot` preprocessing pipeline (`jianingq/scene-graph-robot`), the official code release for *Expanding Spatial and Temporal Context for Robotic Imitation Learning With Scene Graphs* (Qian et al., CVPR 2026). The code implements **Section 3.1, "Task-Driven Scene Graph"**: turning raw DROID wrist-camera episodes into per-frame, object-centric scene-graph tensors. Section 3.2 (the diffusion policy) is a separate stage and is not in this repository.

Every claim below is grounded in the actual functions and variable names. Tensor shapes are for the default configuration (`vit_image_size: 224`, `preprocess_final_size: 448`). Two additions made on the `feature/section3.1-faithful` branch are called out where they plug in: an LLM entity-extraction front-end (`llm_entities.py`) and a DINOv2 backbone option (`dinov2_encoder.py` + a branch in `vit.py`).

2. Architecture at a glance

The whole run is a nested loop: episodes on the outside, frames on the inside, with a single set of models loaded once.

```
droid_hodor_test_cutie.py  ->  post_process_scene_cutie.main(config)
  load_config(path)                     # YAML layering + LLM entity hook
  HODORProcessor(config)                # loads ViT + Cutie/GroundingDINO/SAM once
  crawler(data_path)                    # find episode dirs
  for each episode: process_episode(...)
      load_trajectory(...)              # DROID .h5 + MP4 -> list of timesteps
      for each frame: processor.process_frame(...)
          tracker.process(...)          # detect (GroundingDINO+SAM), propagate (Cutie) -> label_mask
          _mask_to_instances(...)       # label_mask -> per-class binary masks + 2D boxes
          _add_centroids(...)           # depth backprojection -> 3D centroids
          _obtain_features(...)         # ViT -> global + per-object embeddings
          # write each node into final_rp / final_bboxes at its class slot
      np.save(episode_pr.npy, episode_bboxes.npy)
```

Each scene-graph node becomes one row in two arrays: a **768-d embedding** (`final_rp`) and a **7-vector** (`final_bboxes` = 4 box + 3 centroid). Row 0 is the global scene node.

3. Entry and config resolution

`droid_hodor_test_cutie.py` is a thin CLI shim; all logic is in `post_process_scene_cutie.py`. Config layering happens in `load_config`:

```
        task_config = OmegaConf.load(config_path)
        merged = OmegaConf.create({})
        for default in task_config.get("defaults", []):          # e.g. ["base"]
            merged = OmegaConf.merge(merged, OmegaConf.load(default_path))
```

```
merged = OmegaConf.merge(merged, task_config)            # task overrides base
OmegaConf.resolve(merged)
_maybe_resolve_entities(merged, config_dir, config_stem) # <- added: LLM hook
```

Input: a task YAML such as `configs/chicken.yaml`, which declares `defaults: [base]`. **Output:** one merged `DictConfig` where `base.yaml` provides the floor (`perception.*`, `model.*`, `data.*`) and the task file overrides the specifics (`task.classes`, per-class `class_area_ranges`, thresholds).

`_maybe_resolve_entities` is a no-op unless the task sets `entities.source: llm`; that is the seam for the LLM front-end (Section 8). Manual configs pass straight through unchanged.

4. Data loading

`crawler(dirname)` recursively returns every directory containing a `trajectory.h5`; `main` enumerates them so `episode_num` is a stable integer per episode (Cutie uses it as a state key).

`load_trajectory` is the DROID reader. Its output is the key data structure: `timestep_list`, a numpy array of `T` dicts. Each `timestep["observation"]` carries:

- `["image"][f"{cam}_left"]` - one wrist RGB frame, `(720, W, 3)` BGR. `MP4Reader` splits each stereo MP4 in half: `frame[:, :single_width, :]` becomes `_left`, the other half `_right`.
- `["camera_type"]` - the hardcoded serial->type map; type `0` is the wrist camera.
- `["camera_intrinsics"][wrist_key]` - the 3x3 matrix `K`.
- `["camera_extrinsics"][wrist_key]` - a 6-vector `[x,y,z,rx,ry,rz]` or a 4x4.

Non-obvious mechanic: `load_trajectory` drops steps where teleoperation was paused - `step_skipped = not observation["controller_info"]["movement_enabled"]`, and those timesteps are deleted. Frame indices in the output are already filtered. The depth stream is read separately, and `process_episode` stops if it runs shorter:

```
if frame_num >= len(depth_frames):
    print(f"Stopping at frame {frame_num}: depth stream is shorter than trajectory")
    break
```

4.1 Geometric preprocessing

Two functions establish the canonical frames the rest of the pipeline assumes.

```
def _preprocess_wrist_bgr(raw_bgr, final_size):          # raw (720, >=1050, 3)
    image = Image.fromarray(raw_bgr).crop((150, 0, 1050, 720))    # -> 900x720
    image = ImageOps.expand(image, padding, fill=(0, 0, 0))       # pad to 900x900
    image = image.resize((504, 504), BICUBIC).crop(... 448 ...)   # -> 448x448
```

- `wrist_bgr_unrotated` -> `(448, 448, 3)`.
- `wrist_bgr_rotated = np.rot90(unrotated, k=2)` -> `(448, 448, 3)`. The wrist camera is mounted upside-down, so **the tracker and the ViT see the rotated image**.
- `hand_rgb_800` -> `(800, 800, 3)`, the unrotated image at depth resolution.
- `_preprocess_depth`: `depth_mm[:, 150:1050] / 1000` -> pad -> `[50:850, 50:850]` -> `(800, 800)` float32 metres, using the same 150-px horizontal crop as RGB so the two align.

These feed `process_frame(hand_bgr_rotated, frame_num, episode_num, depth, hand_rgb_unrotated, intrinsics, extrinsics)`.

5. The per-frame pipeline

`HODORProcessor.process_frame` orchestrates five stages per frame.

5.1 Tracking to a label mask

```
label_mask = self.tracker.process(hand_bgr_rotated, frame_num)
```

Input: rotated BGR `(448, 448, 3)`, frame index. **Output:** `label_mask` `(448, 448)` int32; each pixel is a class id `0..num_classes-1`, or `background_label = num_classes`.

`CutieTracker` runs one independent Cutie core **per class** (`self.processors[class_index]`). The `self.detected[class_index]` latch is the control flow:

```
if self.detected[class_index]:
    probability = self.processors[class_index].step(rgb)          # propagate only
    tracked = output_prob_to_mask(probability) > 0
    ...
else:
    detected_mask, _, _ = self.detector.detect_class(image_bgr, class_index, occupied_masks)
    self.processors[class_index].step(rgb, seed, objects=[1])     # initialize
    self.detected[class_index] = True
```

Once a class is seen it is **only propagated**; an empty tracked mask means "temporarily absent" and does not reset or re-detect. `_skip_late_detection` (from `skip_non_first_frame`) decides whether a class may be detected after frame 0.

`GroundedSAMDetector.detect_class` is the detect step: GroundingDINO `predict_with_classes([classes[i]])` -> NMS -> per-box filtering by `class_area_ranges`, `confidence_threshold`, and the overlap gate `_max_prior_mask_coverage(box_mask, occupied_masks) > overlap_threshold`. The candidate is compared against each prior class mask **independently** (the maximum, not a union - a union denominator would weaken the filter as more objects appear). The first surviving box is passed to SAM (`multimask_output=False`) -> one boolean mask `(448, 448)`.

5.2 Label mask to instances

```
labels, binary_masks, bboxes = self._mask_to_instances(label_mask)
```

Output layout:

- `labels`: list of class ids actually present, length `K`.
- `binary_masks`: `(1, K, 448, 448)` float - one channel per present class.
- `bboxes`: `(K+1, 7)` float. Row 0 is the global node, prefilled `[0, 1, 0, 1, 0, 0, 0]` (whole image, no centroid); rows `1..K` hold `[row_min/H, row_max/H, col_min/W, col_max/W, 0, 0, 0]` - a normalized 2D box with the centroid slots still zero.

5.3 3D centroids

```
resized = F.interpolate(binary_masks[:, i:i+1], size=(800, 800), mode="nearest")[0, 0]
unrotated_mask = np.rot90(resized.numpy(), k=2)          # undo the 180 deg to match depth
points, _ = self._depth_to_pointcloud(depth, hand_rgb_unrotated, intrinsics, extrinsics, unrotated_
if len(points) > 0:
    bboxes[i + 1, 4:7] = points.mean(axis=0)
```

The mechanic to notice: masks live in the rotated frame (the tracker and ViT saw the rotated image), so before backprojecting against the **unrotated** depth they are rotated back with `rot90(k=2)`. `_depth_to_pointcloud` does pinhole unprojection with a corrected principal point `cx = intrinsics[0, 2] - 150` (the same 150-px crop), masks to valid depth `depth_min_m < z < depth_max_m` inside the mask, then transforms to the world frame via the extrinsics (`R.from_euler("xyz", ...)` for a 6-vector). Output

`points` is `(M, 3)`; the node centroid is `points.mean(0)` written to `bboxes[i+1, 4:7]`.

5.4 Features

```
        frame_repr, dino_full = self._obtain_features(hand_bgr_rotated, binary_masks)
```

Inputs: rotated BGR `(448, 448, 3)`, `binary_masks (1, K, 448, 448)`. Preprocess resizes to `(1, 3, 224, 224)` with ImageNet normalization. The ViT forward returns `global_repr (1, 768)` and `patch_repr (1, N, 768)`, where **N = 196** for the default DINO-v1 patch16 (14x14 grid) or **N = 256** for DINOv2 b14 (16x16 grid). Object features are pooled onto the token grid:

```
        token_side = int(round(token_count ** 0.5))              # 14 (or 16)
        token_masks = F.adaptive_max_pool2d(instance_masks, (token_side, token_side))  # (1,K,14,14)
        token_masks = token_masks.reshape(1, K, token_count)                            # (1,K,196)
        masked = patch_repr.unsqueeze(1).expand(-1, K, -1, -1).masked_fill(
            ~(token_masks.unsqueeze(-1) > 0.5), float("nan"))
        object_repr = masked.nanmean(dim=2)                      # (1,K,768)
        object_repr = torch.where(isnan, zeros, object_repr)     # empty mask -> zeros
```

Output: `torch.cat([global_repr.unsqueeze(1), object_repr], dim=1)` -> `(1, K+1, 768)`. `adaptive_max_pool2d` marks a token if any mask pixel falls in its cell; the `masked_fill` -> `nan` -> `nanmean` sequence averages exactly the masked tokens without a manual count. Features and 2D masks both live in the rotated frame, so they are consistent; only the centroid path un-rotates.

5.5 Writing nodes into the persistent buffers

```
        self.final_rp[0] = frame_repr[0, 0]                    # global node
        self.final_bboxes[1:, :4] = 0.0                        # zero every 2D box this frame
        for instance_index, class_index in enumerate(labels):
            slot = self.slot_by_class[class_index]
            self.final_rp[slot]   = frame_repr[0, instance_index + 1]
            self.final_bboxes[slot] = bboxes[instance_index + 1]
```

This is how Section 3.1's "out of view -> zero the 2D box, keep the 3D centroid" is realized. `final_rp` / `final_bboxes` are allocated **once per episode** in `_reset_episode_state` (called at `frame_num == 0`). Each frame only the 2D box columns `[:, :4]` are zeroed, so the centroid columns `[4:7]` of an unseen class **carry over** from the last frame it was visible. `slot_by_class` is validated in `__init__` to be a bijection onto `1..num_classes`, so a given class always lands in the same row for the whole episode - that is what makes row `s` mean "the same object" over time.

6. The visual backbone

`get_vit` dispatches on the model name. The original path:

```
        if embedding_name in ["...", "dino-vit-b16"]:
            model, hidden_dim = vit_base_patch16(pretrain_path, img_size=img_size, global_pool=global_pool)
```

With `global_pool=True` (how the processor loads it), `extract_feat_all` returns:

```
        x_all = x[:, 1:, :]             # (1,196,768) patch tokens
        x     = x[:, 1:, :].mean(dim=1)  # (1,768) global = mean of patch tokens (CLS dropped)
        return x, x_all
```

So in the default path the "global" node is a mean of patch tokens.

6.1 DINOv2 branch (added)

A branch placed ahead of that block returns a DINOv2 encoder with an identical `(global, patch)` contract, but using the true CLS token as the global node:

```
if embedding_name.startswith("dinov2"):
    model = DINOv2Encoder(embedding_name, img_size=img_size, checkpoint=...)
    return model, DINOV2_HUB[embedding_name][1]   # 768 for b14
```

```
# dinov2_encoder.py
out = self.model.forward_features(x)
cls_token    = out["x_norm_clstoken"]      # (1,768)   -> global scene node
patch_tokens = out["x_norm_patchtokens"]   # (1,256,768) -> masked pooling
return cls_token, patch_tokens
```

Because the contract is identical, `_obtain_features` needs no changes; only `token_side` shifts 14 -> 16 (N goes 196 -> 256). That is why the swap is a drop-in, selected by `model.vit_model: dinov2-vit-b14`.

7. Outputs

`process_frame` returns a `FrameOutput`; `process_episode` stacks them and saves:

- `episode_pr.npy` -> `(T, N+1, 768)` - node embeddings over time, row 0 global.
- `episode_bboxes.npy` -> `(T, N+1, 7)` - `[row_min/H, row_max/H, col_min/W, col_max/W, cx, cy, cz]`: the first four are the normalized 2D box (zeroed when unseen), the last three the persistent world-frame centroid.
- Chicken task only: `episode_pr_seperated.npy` / `..._bboxes_seperated.npy`, where `_update_separated` splits the single “stuffed animal” class into giraffe vs chicken slots by a pixel-height threshold (`vertical_extent > stuffed_extent_threshold_px`).

`N+1` is `num_classes + 1`; for `pineapple_llm` (2 entities) that is `(T, 3, 768)` and `(T, 3, 7)`.

8. The LLM entity front-end (added)

This is the only place the object list stops being hardcoded. The data transform, end to end:

- **Input:** `task.language_instruction = "pick up the pineapple and place it in the bowl"`.
- `extract_entities` calls the provider (`_call_anthropic` / `_call_openai` via `urllib`), then `_parse_entities` -> `["pineapple", "bowl"]`.
- `to_grounding_classes` -> `["pineapple.", "bowl."]`, the dotted GroundingDINO caption format the detector expects (`".".join(words) + "."`, so “yellow pineapple” -> “yellow.pineapple.”).
- `build_slot_by_class(2)` -> `{0: 1, 1: 2}`.

`resolve_entities` wraps that with a JSON cache (`configs/<task>.entities.json`) keyed on the instruction, so only the first run hits the network. `_maybe_resolve_entities` writes the result into the live config:

```
resolved = resolve_entities(cfg, instruction, config_dir)
task.classes = list(resolved["classes"])                    # -> GroundedSAMDetector
if task.get("slot_by_class") is None:
    task.slot_by_class = build_slot_by_class(len(resolved["classes"]))
```

From there everything downstream is identical to a manual config: the detector gets `classes=["pineapple.", "bowl."]`, `HODORProcessor` validates `slot_by_class`, and the frame loop runs unchanged. The design goal is that the LLM step only *produces* `task.classes` and changes nothing else.

Appendix A. File map

File	Role
`droid_hodor_test_cutie.py`	CLI entry; calls `main(config)`
`post_process_scene_cutie.py`	Orchestrator: `load_config`, `HODORProcessor`, `process_episode`
`extract_part_masks_cutie.py`	`GroundedSAMDetector`, `CutieTracker`, `build_tracker`
`droid_utils.py`	DROID readers: `crawler`, `load_trajectory`, `MP4Reader`
`vit.py`	`get_vit`, DINO-v1 ViT-B/16, CLIP path
`dinov2_encoder.py`	DINOv2 wrapper (added)
`llm_entities.py`	LLM entity extraction (added)
`configs/base.yaml`	Shared defaults
`configs/{giraffe,cabinet,pineapple,chicken}.yaml`	Manual task configs
`configs/pineapple_llm.yaml`	LLM-entities example (added)
`configs/giraffe_dinov2.yaml`	DINOv2-backbone example (added)

Appendix B. Shape reference (default config)

Tensor / structure	Shape	Meaning
wrist RGB (raw)	`(720, W, 3)`	one stereo half, BGR
`wrist_bgr_rotated`	`(448, 448, 3)`	tracker/ViT input (180 deg rotated)
`depth`	`(800, 800)`	metres, aligned to unrotated RGB
`label_mask`	`(448, 448)`	per-pixel class id; bg = `num_classes`
`binary_masks`	`(1, K, 448, 448)`	one channel per present class
`bboxes`	`(K+1, 7)`	4 normalized box + 3 centroid, row 0 global
`patch_repr`	`(1, 196, 768)` / `(1, 256, 768)`	ViT patch tokens (v1 / DINOv2)
`frame_repr`	`(1, K+1, 768)`	global + per-object embeddings
`final_rp`	`(N+1, 768)`	per-episode node embeddings, row 0 global
`final_bboxes`	`(N+1, 7)`	per-episode boxes + persistent centroids
`episode_pr.npy`	`(T, N+1, 768)`	embeddings over the episode
`episode_bboxes.npy`	`(T, N+1, 7)`	boxes + centroids over the episode